UNIT III:
MESSAGE AUTHENTICATION AND HASH FUNCTIONS
LECTURE HOURS- 6 HRS

3.1. Message Authentication

3.2. Hash Functions

3.3. Message Digests: MD4 and MD5

3.4. Secure Hash Algorithms: SHA-1

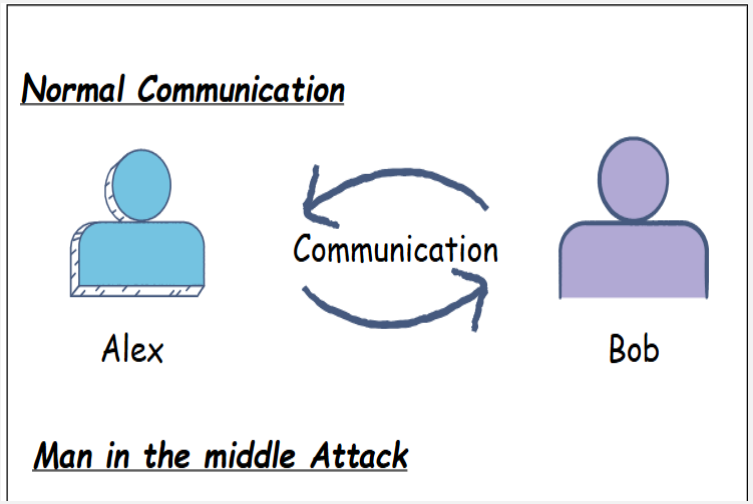
3.5. HMAC

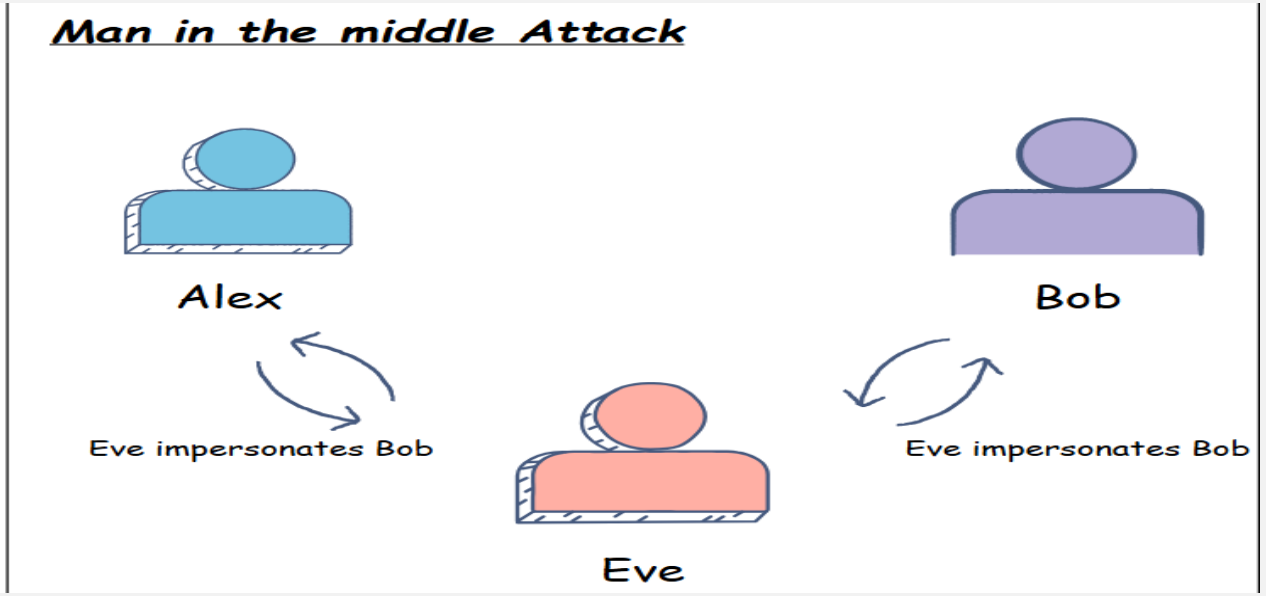
3.6. Digital Signatures

MESSAGE AUTHENTICATION

- **Message authentication** is the security process that ensures a received message is **genuine**, **unchanged**, and **verifiably sent by the claimed originator**.
- **Message authentication** ensures that a received message is:
  i. **Intact** → not modified in transit (**integrity**)
  ii. **Genuine** → really sent by the claimed sender (**origin authentication**)
  iii. **Provable** → sender cannot later deny sending it (**non-repudiation**)
- Encryption alone is **not enough**. Encryption hides content (confidentiality), but an attacker can still **alter**, **replay**, or **forge** messages unless authentication is used.
- Without authentication, an attacker (e.g., "Man-in-the-Middle") can alter the message undetected.

MESSAGE AUTHENTICATION





SECURITY REQUIREMENTS OF MESSAGE AUTHENTICATION

- A secure system should resist:

Attack	What attacker tries	What we must ensure
Masquerade	Pretend to be the sender	Verify sender identity
Modification	Change message content	Detect any change
Replay	Re-send old message	Detect duplicates/old messages
Reordering	Change message order	Preserve sequence integrity
Repudiation	Sender denies sending	Provide proof of origin

THREE WAYS TO ACHIEVE MESSAGE AUTHENTICATION

1) Using Message Encryption
2) Using a Message Authentication Code (MAC)
3) Using a Hash Function

1) USING MESSAGE ENCRYPTION

- Encryption can also provide authentication **if used properly**.
- Sender encrypts the message with a key.
- Receiver decrypts it with the same/shared (or corresponding) key.
- If decryption works and content makes sense, it likely came from the sender.

Example with symmetric key

- Alice and Bob share key **K**
- Alice sends: C = E(K, M)
- Bob decrypts: M = D(K, C)
- If Bob successfully decrypts, he assumes it came from Alice (since only she has **K**).

Pros

- Provides confidentiality + some authentication

Cons

- Inefficient for large data if only authentication is needed
- Does **not** give non-repudiation (both share the same key)

2) USING A MESSAGE AUTHENTICATION CODE (MAC)

A **MAC** is a short tag generated from the message **and a secret key**.

Process

- Sender computes: MAC = F(K, M)
- Sends: (M, MAC)
- Receiver recomputes MAC using same key and compares
- If MACs match → message is authentic and unmodified.

Common MAC algorithms

- HMAC
- CMAC

Properties

- Ensures integrity
- Ensures origin authentication
- Fast and efficient
- No confidentiality (message is visible)

Limitation

- No non-repudiation (both parties share Key)

3) USING A HASH FUNCTION

- A hash function produces a fixed-size **digest** from any message.

$$h = H(M)$$

- But **hash alone is NOT authentication**. Anyone can compute it. So, hashes are combined with keys or signatures:

(a) Hash + Secret Key (HMAC style)

$$MAC = H(K \,||\, M)$$

(b) Hash + Digital Signature (for non-repudiation)

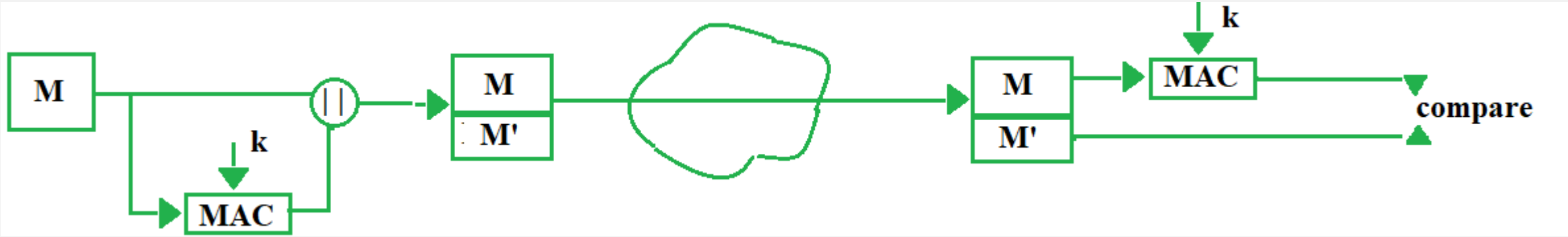
i. Sender computes hash: $h = H(M)$
ii. Encrypts hash with **private key** → digital signature
iii. Receiver verifies with **public key**

This provides:

- ✓ Integrity
- ✓ Authentication
- ✓ Non-repudiation

MESSAGE AUTHENTICATION CODE (MAC)

- A Message Authentication Code (MAC) is a cryptographic technique used to ensure that a message has not been altered and that it comes from the legitimate sender.
- A MAC works by using a secret key that is shared between the sender and the receiver.
- The system requires four components: the message, the secret key, a MAC algorithm such as **HMAC** or **CMAC**, and the MAC value (tag).



MESSAGE AUTHENTICATION CODE (MAC)

Sender Side

- The sender takes the original message that needs to be transmitted.
- The sender applies the MAC algorithm to the message using the shared secret key.
- The MAC algorithm produces a fixed-size value known as the MAC tag.
- The sender sends both the message and the MAC tag to the receiver.

Receiver Side

- The receiver receives the message along with the MAC tag.
- The receiver uses the same secret key and the same MAC algorithm to compute a new MAC value from the received message.
- The receiver compares the newly generated MAC value with the MAC tag received from the sender.

MESSAGE AUTHENTICATION CODE (MAC)

Result of Verification

- If both MAC values are the same, the receiver accepts the message as authentic and unmodified.
- If the MAC values are different, the receiver rejects the message because it has either been altered or sent by an unauthorized sender.

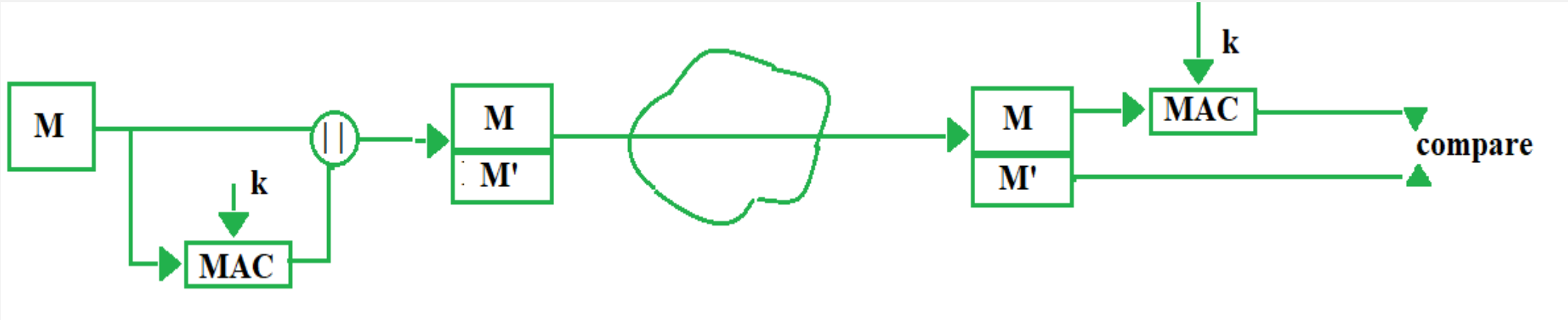
What MAC Provides

- A MAC provides message integrity by detecting any change in the message.
- A MAC provides authentication by confirming that the sender knows the secret key.
- A MAC does not provide confidentiality because the message is transmitted in readable form.

MODELS OF MESSAGE AUTHENTICATION CODE

There are different types of models of Message Authentication Code (MAC) as follows:

- **MAC without encryption -** This model can provide authentication but not confidentiality as anyone can see the message.



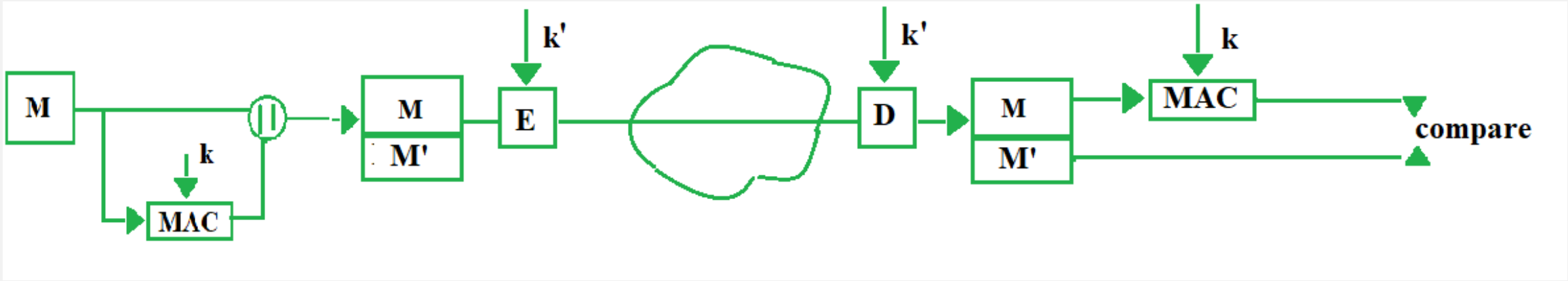
ENCRYPT-THEN-MAC

1. Sender **encrypts the message** for confidentiality.
2. Sender creates MAC from the **original message**.
3. Sender sends **encrypted message + MAC**.

At receiver:

1. Receiver **decrypts first**.
2. Then computes MAC from the decrypted message.
3. Compares with received MAC.

- **Problem:**
  If the message was altered, the receiver **wastes time decrypting** a bad message.


M
k
MAC
M
M'
k'
E
k'
D
M
M'
k
MAC
compare

ENCRYPT-THEN-MAC

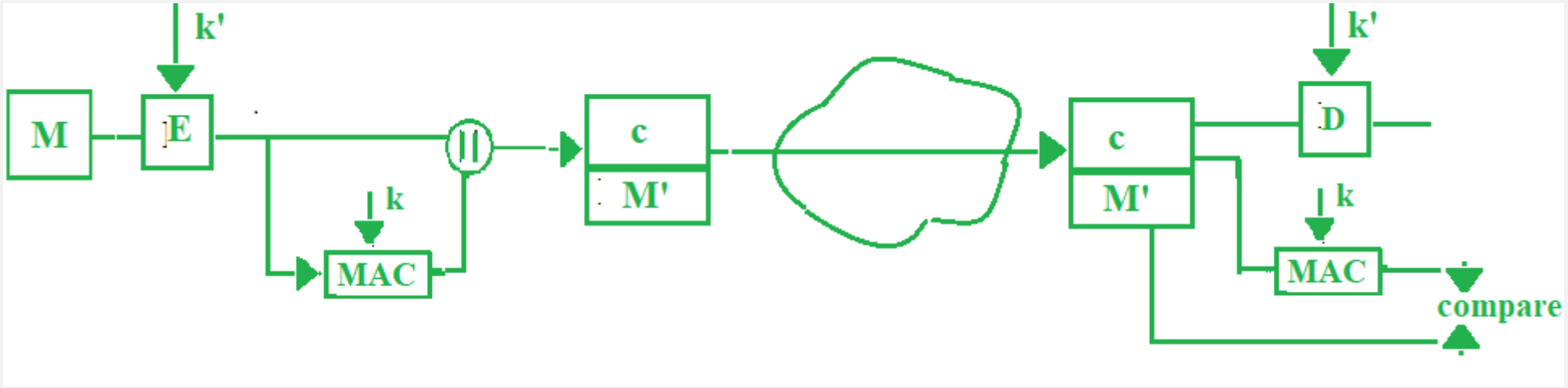
1. Sender encrypts the message:
$$c = E(M, k')$$
2. Sender creates MAC from the **encrypted message**:
$$M' = MAC(c, k)$$
3. Sender sends **c + MAC**.

At receiver:

1. Receiver **checks MAC first** using c.
2. If MAC matches → **then decrypt**.
3. If MAC does not match → **discard immediately**.

Advantage:

- No time wasted decrypting a tampered message.


k'
M
E
k
MAC
c
M'
c
M'
k'
D
k
MAC
compare

HASH FUNCTION

- A **hash function** is a mathematical function that turns **any size input** into a **fixed-size output** called a **hash** or **digest**.

$h = H(M)$ where,

M =message (any length)

h= hash value (fixed length)

Example:

- H("hello") = 2cf24dba5fb0a...
- H("hello!") = ce06092fb948...

- A **small change** in input creates a **completely different** hash.

HASH FUNCTION...

What a hash function does

- No matter how big the input is, the output size is the same.
- Examples of cryptographic hash functions:
  - **SHA-256** → 256-bit output
  - **SHA-1** (now considered weak)
  - **MD5** (broken for security use)

How Hash Values Are Calculated

- Hash functions do **bitwise operations**, **modular arithmetic**, **shifting**, **mixing**, and **compression** repeatedly.
- They process data in **blocks** and update an internal state.

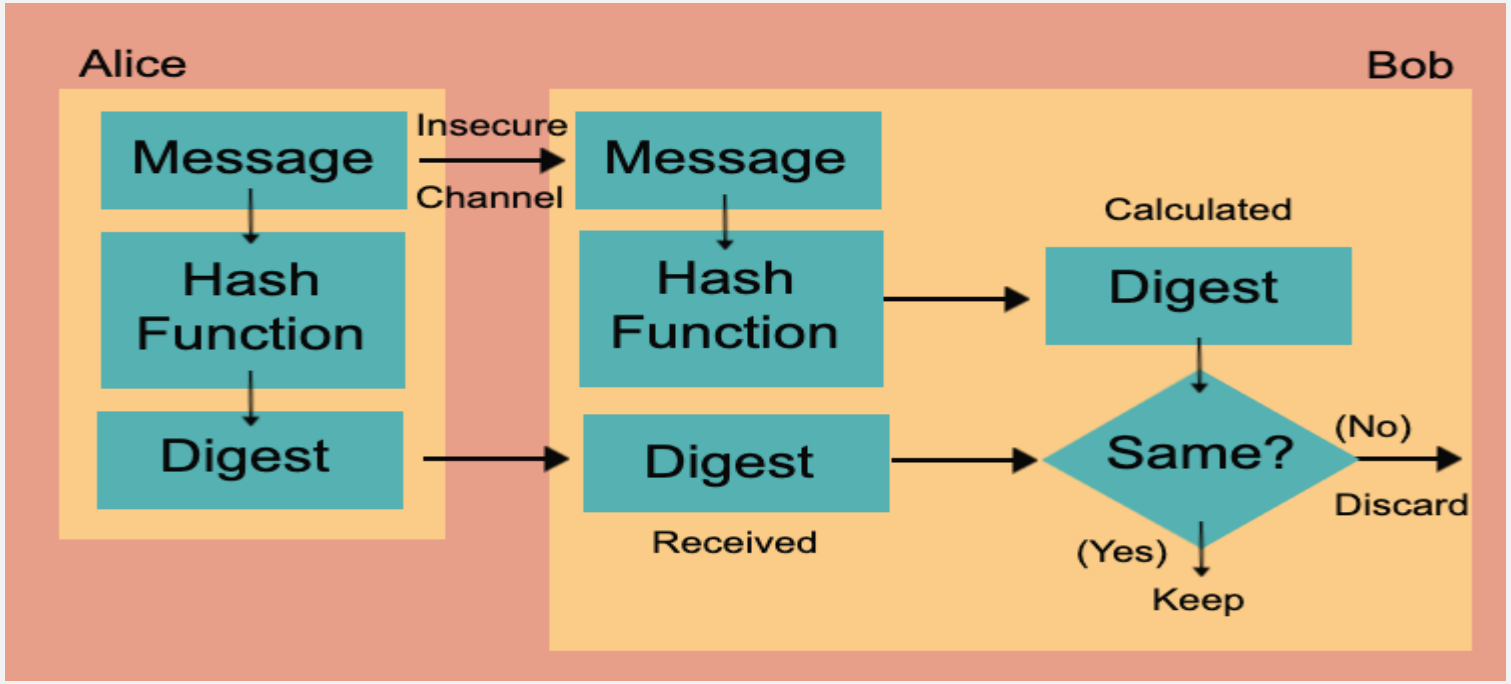
PROPERTIES OF A GOOD HASH FUNCTION

A good (especially cryptographic) hash function has these properties:

a. **Deterministic:** Same input → same hash every time.
b. **Fixed Length Output:** No matter how big the input is, output size is constant.
c. **Fast to Compute:** Hashing should be quick.
d. **Pre-image Resistance:** Given a hash **h**, it should be **very hard** to find Message **M**.
e. **Collision Resistance:** Very hard to find **any two** different inputs with same hash.
f. **Avalanche Effect:** A tiny change in input → huge change in output.

MESSAGE DIGEST

- A **message digest** is the fixed-length output produced by applying a cryptographic hash function such as **MD5**, **SHA-1**, or **SHA-256**. to a message.
- Any size input → fixed size output (digest)



MESSAGE DIGEST…

Uses of Message Digest

a. Integrity Check

- Sender sends:
- Message + Digest
- Receiver recomputes the digest and compares.

b. Digital Signatures

- The digest (not the whole message) is signed for efficiency.

c. Password Storage

- Systems store the digest of passwords, not the passwords.

d. Message Authentication Code (MAC)

- Digest is combined with a secret key to verify sender authenticity.

MD4

- **MD4 (Message Digest 4)** is an early cryptographic hash function designed by **Ronald Rivest** in 1990.
- It produces a **128-bit message digest** from any input.

Why MD4 was created

- MD4 was built to be **very fast** on 32-bit computers and to show how practical hash functions could be for integrity checks.

Key Properties

- Output size: **128 bits**
- Very **fast**
- Simple design (3 rounds of operations)
- Early foundation for later hashes

MD4

⚙️ Steps of the MD4 Algorithm

1. Input Message

Start with any message

Example: `"I love you"`

---

2. Padding

- Append a single `1` bit
- Add `0` bits until message length ≡ **448 mod 512**
- Same padding rule as MD5

---

3. Append Length

- Append **64-bit representation of original message length**
- Final size becomes a multiple of **512 bits**

---

4. Initialize Buffers

MD4 uses 4 registers (same as MD5):

```
A = 0x67452301
B = 0xEFCDAB89
C = 0x98BADCFE
D = 0x10325476
```

5. Process Each 512-bit Block

MD4 has **3 rounds (not 4 like MD5)**

Each round has **16 operations → total 48 operations**

🔁 Functions Used

- **F(B,C,D)** = (B AND C) OR (NOT B AND D)
- **G(B,C,D)** = (B AND C) OR (B AND D) OR (C AND D)
- **H(B,C,D)** = B XOR C XOR D

🔁 Rounds Overview

- **Round 1:** Uses function F
- **Round 2:** Uses function G
- **Round 3:** Uses function H

Each operation involves:

- Addition
- Bitwise operations
- Left circular shifts

6. Update Buffers

After each block:

```
A = A + previous A
B = B + previous B
C = C + previous C
D = D + previous D
```

---

7. Final Output

- Combine A, B, C, D
- Produce **128-bit hash value**

🧪 Example

Input:

```
"I love you"
```

MD4 Hash:

```
85c8bce8f6b9b8a3d8c9c2f6c9f5c3a1
```

STEPS OF THE MD4 ALGORITHM

i. **Input Message**
The algorithm accepts a message of any length (in bits).

ii. **Padding the Message**
A single 1 bit is appended to the message, followed by enough 0 bits so that the total length becomes **448 mod 512** (i.e., 64 bits short of a multiple of 512).

iii. **Appending Message Length**
The original length of the message (before padding), represented as a **64-bit binary value**, is appended.
Now, the total message length becomes a multiple of **512 bits**.

iv. **Initialize Buffers**
Four 32-bit registers are used: **A, B, C, and D**.
These are initialized with fixed hexadecimal constants.

v. **Block Division**
The padded message is divided into **512-bit blocks**.
Each block is further split into **16 words**, each of **32 bits**.

STEPS OF THE MD4 ALGORITHM

vi. Processing Each Block

Each 512-bit block is processed sequentially using **three rounds of operations**.

These operations update the values of A, B, C, and D using the block's 32-bit words.

Round 1 (uses F)

For each operation: $A = (A + F(B, C, D) + M[k]) <<< s$

<<< s = left rotate by s bits.

Round 2 (uses G)

- Adds a constant: $A = (A + G(B, C, D) + M[k] + 0x5A827999) <<< s$

Round 3 (uses H)

- Adds another constant: $A = (A + H(B, C, D) + M[k] + 0x6ED9EBA1) <<< s$

Round Functions

MD4 uses three simple logical functions:

- F(X,Y,Z) = (X AND Y) OR (NOT X AND Z)
- G(X,Y,Z) = (X AND Y) OR (X AND Z) OR (Y AND Z)
- H(X,Y,Z) = X XOR Y XOR Z

Each round uses one of these.

THE PROBLEM WITH MD4

- MD4 is **cryptographically broken**.
- Researchers quickly found **collisions** (two different messages giving the same digest). Because of this:
  - It is **not secure**
  - It should **never** be used for security today

What came after MD4

- Because MD4 was weak, Rivest improved it into:
- **MD5** (still later broken)
- Then the industry moved to **SHA-1** and later **SHA-256**

MD5

- **MD5** is a widely known **cryptographic hash function** that takes any input (text, file, password, etc.) and converts it into a **fixed 128-bit hash value** (usually shown as a 32-character hexadecimal number).

What MD5 Does

- Takes input of **any length**
- Produces a **fixed-length output (128 bits)**
- Even a tiny change in input → completely different hash
- Used for **data integrity checks** (but NOT secure for cryptography today)

MD5

⚙️ Steps of the MD5 Algorithm

1. Input Message

Start with any message
Example: `"I love you"`

---

2. Padding the Message

- Add a `1` bit (10000000 in binary)
- Then add `0` bits until message length ≡ **448 mod 512**
- This ensures message is **64 bits short of 512**

---

3. Append Length

- Add **64-bit representation of original message length**
- Now total length becomes a multiple of **512 bits**

---

4. Initialize Buffers

MD5 uses **4 registers (32-bit each)**:

```
A = 0x67452301
B = 0xEFCDAB89
C = 0x98BADCFE
D = 0x10325476
```

5. Process in 512-bit Blocks

Each block goes through **4 rounds**, each with 16 operations (total = 64 operations)

🔁 **Four Functions Used:**

- **F(B,C,D)** = (B AND C) OR (NOT B AND D)
- **G(B,C,D)** = (B AND D) OR (C AND NOT D)
- **H(B,C,D)** = B XOR C XOR D
- **I(B,C,D)** = C XOR (B OR NOT D)

Each round:

- Uses one of these functions
- Performs:
  - Addition
  - Bitwise operations
  - Left rotations

---

6. Update Buffers

After processing each block:

```
A = A + previous A
B = B + previous B
C = C + previous C
D = D + previous D
```

7. Final Output

Combine A, B, C, D → produce **128-bit hash**

SHA -1

- **SHA-1 (Secure Hash Algorithm 1)** is a **cryptographic hash function** that takes an input of any length and produces a **fixed 160-bit (20-byte) hash value**, usually shown as a 40-digit hexadecimal number.

Basic Idea

- Input: Any message (e.g., "I love you")
- Output: Fixed-size hash (160 bits)
- Property: Even a tiny change in input → completely different hash

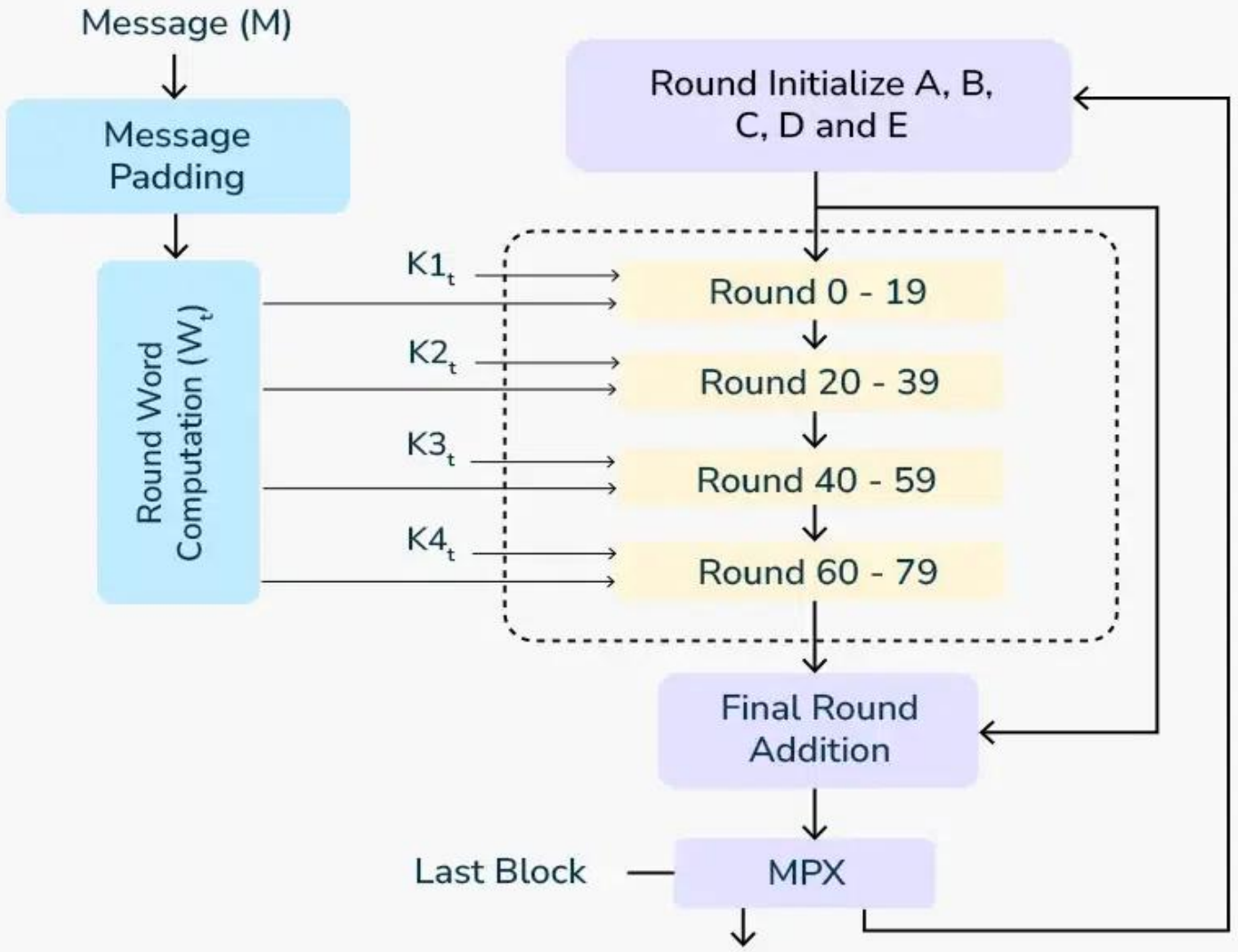
HOW SHA-1 WORKS

- The block diagram of the SHA-1 (Secure Hash Algorithm 1) algorithm. Here's a detailed description of each component and process in the diagram:

Process Flow

- **Message (M):** The original input data that needs to be hashed.
- **Message Padding:** The message is padded so its length becomes 448 modulo 512, making it ready for block processing.
- **Word Computation:** The padded message is divided into 512-bit blocks, each split into 16 words, which are then expanded into 80 words.
- **Initialization:** Five working variables A, B, C, D, and E are initialized with predefined constant values.
- **Round Constants:** Four constants (K1 to K4) are used across different round ranges (0–19, 20–39, 40–59, 60–79).
- **80 Rounds Processing:** The algorithm performs 80 iterations, applying logical operations and transformations on A–E using the expanded words and constants.
- **Final Addition:** The results of the rounds are added to the initial values of A–E to form the intermediate hash.
- **Message Digest (MPX):** All values are combined to produce the final 160-bit hash output.

SHA-1 Algorithm Block Diagram



HMAC (HASH-BASED MESSAGE AUTHENTICATION CODE)

- **HMAC** is a way to use a **hash function** together with a **secret key** to provide:
  - ✅ **Integrity** (message not changed)
  - ✅ **Authentication** (came from someone who knows the key)
- It works with hashes like **SHA-256**, **SHA-1**, etc.

Why a plain hash is not enough

- If you send a message with its hash:
- M, hash(M)
- an attacker who intercepts it can **modify the message** and simply **recalculate the hash** for the altered message.
- Because a normal hash uses **no secret**, anyone can generate a valid hash for any content.
- To prevent this, a **secret key** must be included in the hashing process — which is exactly what **HMAC** does.

HMAC FORMULA

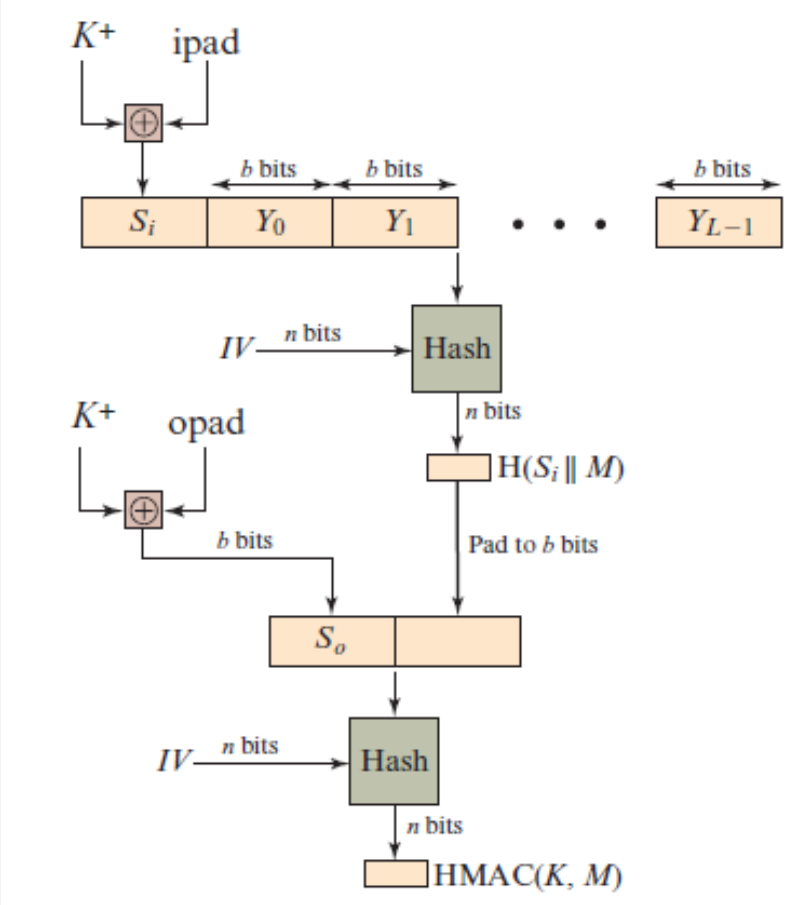
$$\text{HMAC}(K, M) = H((K' \oplus opad) \;||\; H((K' \oplus ipad) \;||\; M))$$

Where:

- H= hash function (e.g., SHA-256)
- K= secret key
- K' =key padded to block size
- ipad = inner padding = 0x36 repeated
- opad = outer padding = 0x5c repeated
- || = concatenation
- $\oplus$ = XOR

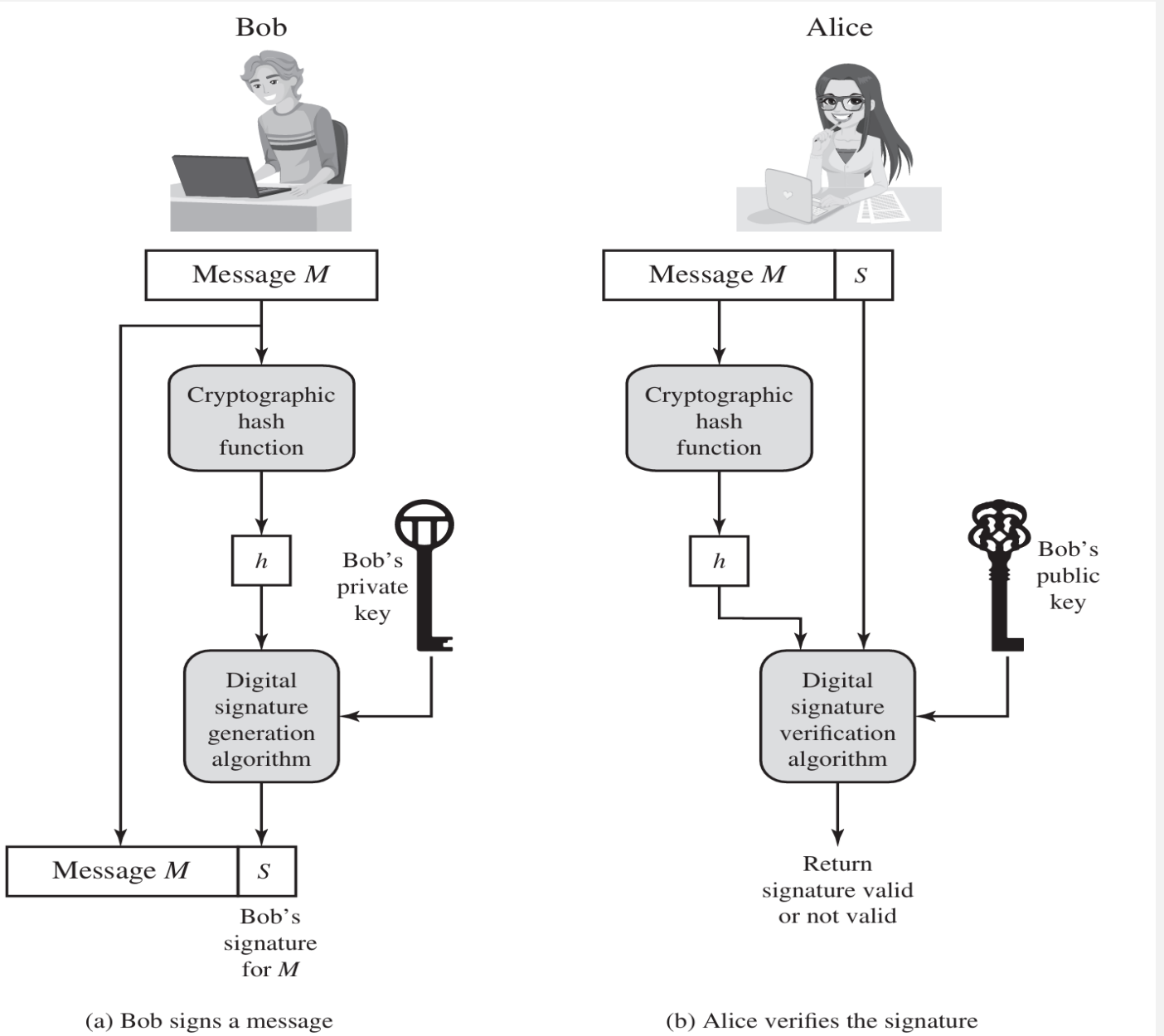
Advantages

- HMACs suit high-performance systems such as routers due to fast computation and verification using hash functions compared to public key methods.
- Digital signatures are larger in size, while HMACs offer comparable security with better efficiency.
- HMACs are preferred in environments where public key systems are restricted or not allowed.
- Lower computational overhead makes HMACs suitable for resource-constrained systems.
- Provides strong message integrity and authentication in real-time communication systems.



DIGITAL SIGNATURE

- A **digital signature** is a cryptographic technique used to verify the **authenticity**, **integrity**, and **non-repudiation** of a digital message or document.
- It ensures that the message was created by a known sender and that it has not been altered during transmission.



(a) Bob signs a message

(b) Alice verifies the signature

Figure 2.7 Simplified Depiction of Essential Elements of Digital Signature Process

DIGITAL SIGNATURE WORKING…

Step-1 Key Assignment:

- Every user has a **key pair**:
  i. **Private key (Signature key)** → used for **signing**
  ii. **Public key (Verification key)** → used for **verification**
- These keys are usually **different from encryption/decryption keys.**

Step-2 Signing Process (Sender Side)

i. The sender takes the **original data/message**
ii. Applies a **hash function** (like SHA-256) and produces a **fixed-length hash value (message digest).**
iii. The hash is then encrypted using the sender's **private key** via a **signature algorithm** (e.g., RSA or DSA)
iv. This produces the **digital signature**
v. The sender sends:
   - Original data
   - Digital signature

DIGITAL SIGNATURE WORKING…

Step 3 — Receiver verifies the signature

- The receiver does two things:
- Decrypts the **digital signature** using the sender's **public key** → gets $Hash_1$
- Hashes the received message again using SHA-256 → gets $Hash_2$

Step 4 — Compare hashes

- If $\mathbf{Hash_1 = Hash_2}$ → message is authentic and unchanged
- If not → message was tampered with or not from the claimed sender